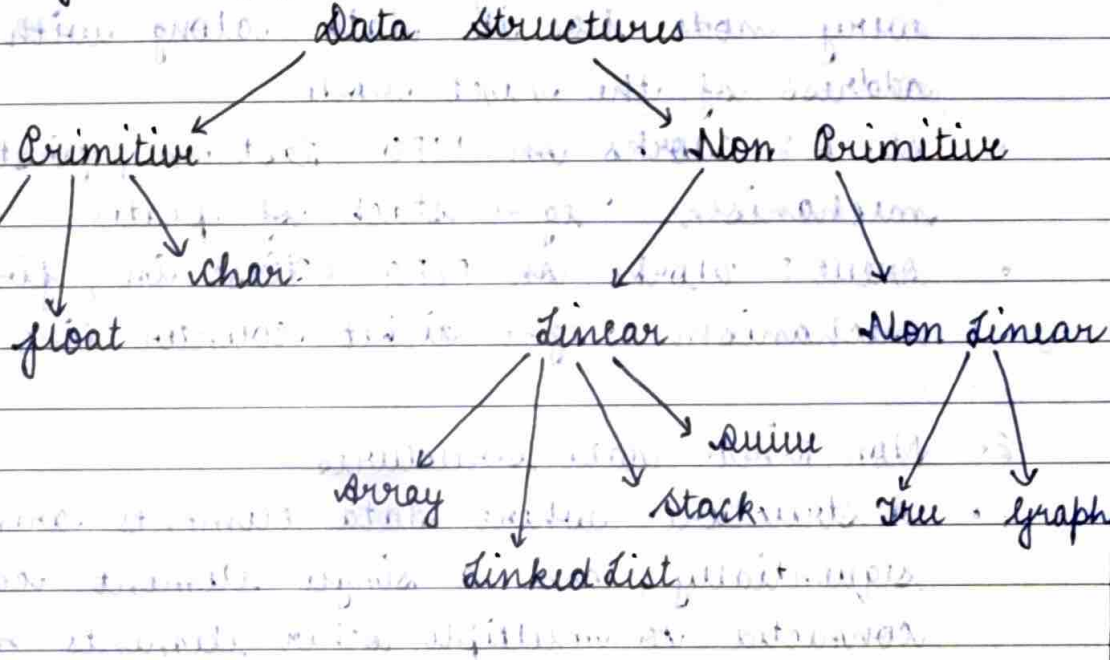
Unit - 1

→ Data Structure

- A data structure is a specialized format, logical organisation and mathematical model used for organising, processing, retrieving and storing data in a computer's memory so that operations can be performed efficiently

→ Classification of data structures



→ Primitive data structures

- They are the fundamental, pre-defined data types directly supported by machine architecture and programming languages to hold single atomic values
- eg → int, float, char, boolean, pointer

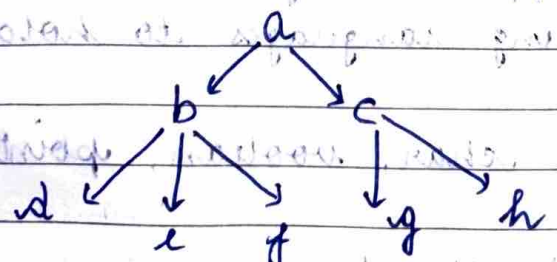
→ Non Primitive data structures

- They are the combinations of primitive data types and are complex, user-defined data structures derived from primitive data types

that can store collections of homogeneous or heterogeneous data items.

1. Linear data structures
   - A linear data structure is the structure in which data items are arranged sequentially or linearly, where each element is attached to the previous & next (adjacent) element
   - Array : contiguous (continuous) memory block of fixed size and same data types
   - Linked list : collection of dynamic nodes where every node has it's data along with memory address of the next node
   - stack : works on LIFO (last in, first out) mechanism : eg → stack of plates
   - queue : works on FIFO (first in, first out) mechanism : eg → ticket counter line

2. Non linear data structures
   - A structure where data elements aren't arranged sequentially, and a single element can be connected to multiple other elements representing hierarchial/ network relationships
   - Tree : Hierarchial structure like parent child relations



   - Graph : Network of nodes (vertices) and connections (edges)

→ Data structures operations

- Actions that can be taken on the data structure are known as data structure operations. It includes :-

1. Traversing : Accessing or visiting each element of a data structure exactly once in order to process / display it
2. Insertion : The process of adding a new data element at a specific position within the data structure
3. Deletion : The process of removing an existing data element from a specified location in the data structure
4. Searching : Finding the location / memory address of a target element from a data structure
5. Sorting : Arranging the elements in a specific logical sequence either in ascending / descending order
6. Merging : Combining the elements of two separate data structures into a single consolidated data structure

→ Algorithm complexity

- There are many ways (algorithms) to solve a problem, but to find the best method we check how much time it takes (Time complexity) and the space it takes (space complexity) based on the number of steps / operations increased as the input size (n) is increased.
- Algorithm complexity is a measure of amounts of computational resources (time & space) that an algorithm requires as a function of input size (n)

1. Time complexity

- The increase in amount of time taken increases as a function of input size (n)

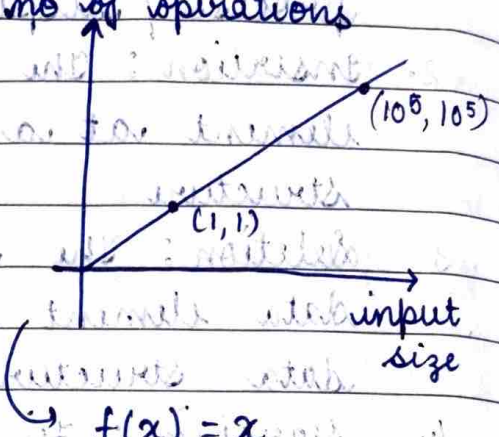
for eg → linear search of array

```
for (i=0; i<n; i++) {
    if (arr[i] == target) {
        return i;
    }
}
```

n = 1, loop runs 1 times

n = 10 ⟶ 10 times

$n = 10^5$ ⟶ 10^5 times



⇒ $f(n) = n$

i.e. $O(n)$

→ Big O Notation

- gives the no of atmost operations required in worst case scenario (upper bound)

→ How to find time complexity?

1. Ignore constants
2. Consider largest term

⇒ If $f(n) = 5n^2 + \sqrt{n} + 100$

⇒ $1 + n^2 + \sqrt{n}$

⇒ Time complexity = $O(n^2)$

→ Big omega Notation (Ω)

- Represents the lower bound of an algorithm's running time i.e. the best case scenario or minimum number of operations

→ Big Theta Notation (Θ)

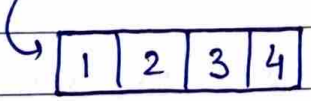
- It defines a tight bound on the execution time of an algorithm bounding it from both above and below
- It considers the average case scenario

2. Space Complexity

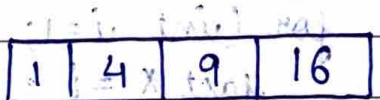
- Amount of space taken by an algorithm as a function of input size (n)

for eg :-

1. input → array []

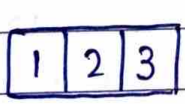


output → array of squares



O(n)

2. input → array []

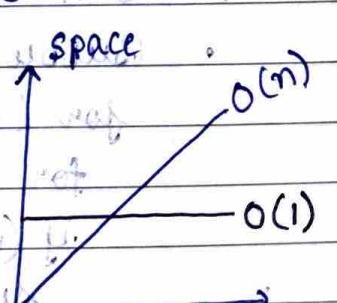


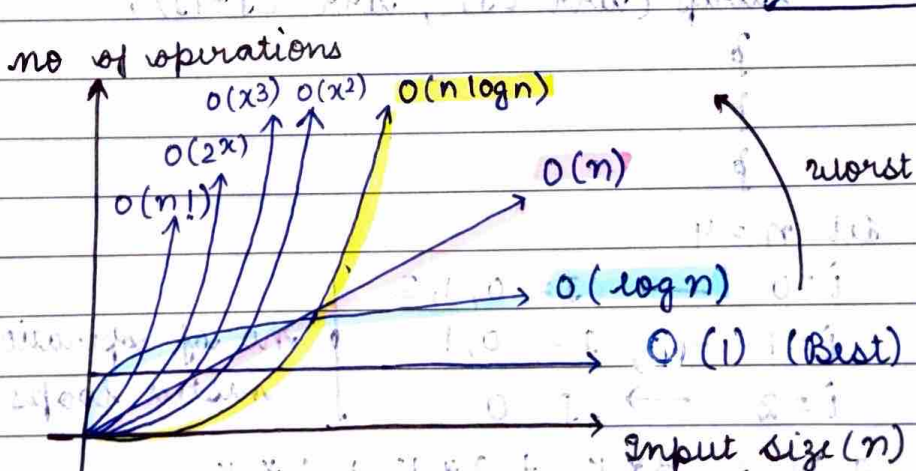
output → sum of elements

1+2+3 = 6

O(K)

i.e. : O(1)





TIME COMPLEXITY

$O(1) > O(\log n) > O(n) > O(n \log n) > O(x^2) > O(x^3) > O(2^x) > O(n!)$

Best case

Worst case

→ common time complexities

1. O(1) → CONSTANT
- Sum of numbers from 1 to n

```
int n;
int ans = n * (n+1) / 2;
```

→ whatever is value of n, the formula is same & is executed in same no of steps

2. O(n) → LINEAR
- Factorial of a no.

```
int fact = 1;
for (int i = 1; i <= n; i++) {
    fact *= i;
}
```

↳ depends on n

- Fibonicci series
- Kadane's algorithm

3. O(n²) & O(n³)
- Bubble sort

```
for (i = 0; i < n-1; i++) {
  for (J = 0; J < n-i-1; J++) {
    if (arr[J] > arr[J+1]) {
      swap (arr[J], arr[J+1]);
    }
  }
}
```

let n = 4

i = 0 → J = 0, 1, 2
i = 1 → J = 0, 1
i = 2 → J = 0

} no of operations in nested loops

→ TC = 3 * K + 2 * K + 1 * K

For n, TC = (n-1) * K + (n-2) * K . . . . 1 * K

$\Rightarrow$ $TC = \frac{n(n-1)K}{2} = \frac{n^2K}{2} - \frac{n}{2}$

$\Rightarrow$ $TC = O(n^2)$

- $O(n^3)$ → Three nested loops, for eg to print all the sub arrays of an array.

4. $O(\log n)$

- Binary search

s → start of array
e → end of array

```
int s = 0, e = n-1;
while (s <= e) {
int mid = s + (e-s)/2;
if (arr [mid] < target) {
    s = mid + 1; }
else if (arr [mid] > target) {
e = mid - 1; }
else {
    return mid;
    }
}
```

$\frac{n}{2^0} \longrightarrow \frac{n}{2^1} \longrightarrow \frac{n}{2^2} \longrightarrow \frac{n}{2^x} = 1$

$\frac{n}{2^x} = 1$

$\Rightarrow \log n = x \log 2$

$\Rightarrow x = \log_2 n$

$T.C = O(\log n)$

5. $O(n \log n)$

- Sortings → merge sort, quick sort
- greedy algorithm

→ Abstract data type (ADT)

- It's a mathematical model / specifications for data types where the data type is defined by it's behaviour from the point of view of a user, specifically in terms of possible values and possible operations, independently of any specific implication
- eg → while driving a car, we know that pressing brakes would stop the car, but we don't know the mechanism of hydraulics that happens in engine to stop the car as its hidden from us (users). Here, the brake pedals are ADT interface

→ Advantages of ADT

1. Abstraction & information hiding
   - Complex low level unrequired implementation details are hidden from the user
2. Reusability & Modularity
   - Once ADT contract is implemented, the source code could be changed multiple times without effecting the main application
3. Flexibility to change implementation
   - If we implement stack using array today and found that it's easier to do the same using linked list tomorrow so we can directly change the main code

→ Components of ADT

1. Declaration of data / values
   - It tells what to store

2. Declarations of operations (interface)
- Includes function definitions, input parameters & return types

3. Axioms / Rules
- Includes the pre & post conditions required for the operation to be performed
- eg → we can't use pop() operation if stack is empty

→ Difference between data type, ADT & Data structure

1. Data type
- At the programming language level
- Focuses on memory representation & built in operations
- It's pre-defined in compiler
- eg → int, float, double

2. ADT
- Based on theoretical / specification level
- Focuses on what operations can be performed
- visibility is based on conceptual / class interface
- eg → stack ADT, list ADT, queue ADT

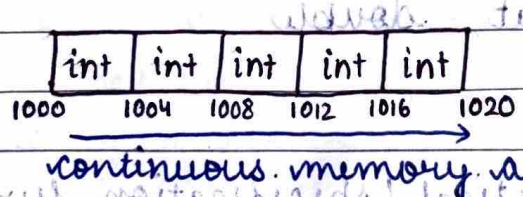
3. Data structure
- Based on implementation level
- Focuses on how data is organised in memories and algorithms used.
- It's visible in actual source code
- eg → array based stack, doubly linked list and binary search tree

→ Standard examples of ADT

1. List ADT
   - insert(), delete(), get(), size()
2. Stack ADT
   - push(), pop(), peak()/top(), isEmpty()
3. Queue ADT
   - enqueue(), dequeue(), front(), isEmpty()

→ Array

- An array is a linear data structure that contains a collection of homogenous (similar data types) elements stored at continuous memory locations where each element can be randomly accessed using an integer index



→ Advantages of Array

- Random access to any element with a time complexity O(1) which is very good.
- It provides memory efficiency for data because the memory is continuous, so we don't need to take up space for pointers or addresses (as in linked list)
- It's cache friendly cause CPU pre fetching is very fast due to continous memory allocation

→ Applications of Array

- It helps in implementation of other structures as it acts as basic structure for stack, queue heaps, hash tables & matrices.
- It canbe used for look up tables as data retrieval is very fast
- It can be used to perform sorting algorithms like quick sort, bubble sort, merge sort etc

→ Implementation of Array

```
# include < stdio.h >
# define N 5

void display (int arr [], int size) {
if (size == 0) {
printf ("Array is empty");
   }
else {
printf ("Array elements :");
for (int i = 0; i < size ; i++) {
printf ("%d\t", arr [i]);
  }
 }

int insert (int arr [], int * size, int element,
                    int pos) {
 if (* size >= N) {
 printf("Array is full");
    }
else if ( pos < 0 || pos > * size) {
printf ("error ! invalid insertion");
   }
```

```
for (int i = *size; i > pos; i--) {
arr[i] = arr[i-1];
}
arr[pos] = element;
(*size)++;
}
int delete (int arr[]; int *size, int pos){
if (*size == 0) {
printf ("underflow");
}
else if (pos < 0 || pos >= *size) {
printf ("invalid position");
}
for (int i = pos; i < *size - 1; i++) {
arr[i] = arr[i+1];
}
(*size)--;
}
int main (){
int arr[N];
int size = 0, choice, element, pos;
printf ("enter initial no of elements");
scanf ("%d", &size);
if (size < 0 || size > N) {
printf ("invalid size");
}
else {
printf ("enter %d elements \n", size);
for (int i = 0; i < size; i++) {
scanf ("%d", &arr[i]);
}
}
```

```
while (1) {
printf ("operations Menu :-");
printf ("1. display");
printf ("2. insert");
printf ("3. delete");
printf ("4. exit");
printf ("enter your choice from 1 to 4");
scanf ("%d", &choice);

switch (choice) {
case 1:
 display (arr, size);
 break;

case 2:
printf ("enter element to insert");
scanf ("%d", element);
printf ("enter position (0 to %d)", size);
scanf ("%d", pos);

if (insert (arr, *size, element, pos)) {
printf ("element inserted successfully");
 }
break;

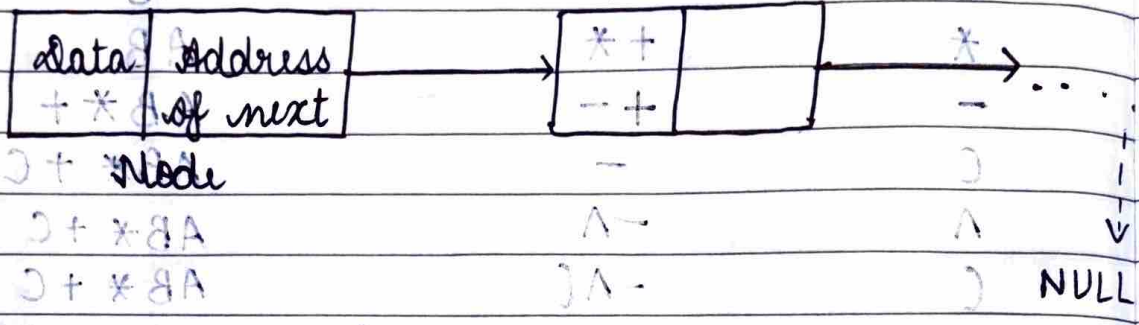
case 3:
printf ("enter position to delete (0 to %d)", size-1);
scanf ("%d", &pos);
if ( delete (arr, *size, pos)) {
printf ("element deleted successfully");
 }
break;
```

```
case 4:
printf ("exiting program");
exit 0;

Default:
printf ("invalid choice, choose between 1 to4");
}
}
return 0;
}
```

→ Linked List

- A linked list is a linear data structure where elements are stored in non contiguous memory locations and linked together using pointers



- It uses dynamic memory allocation of heap memory
- Linked lists are dynamic and grow & shrink at runtime

→ Advantages of linked lists

- Dynamic memory allocation prevents memory waste
- Fast insertion & deletion without element shifting
- Effective execution for building stacks & queue

→ Disadvantages of linked lists

- No random / indexed access
- Extra memory overhead for storing pointers
- Unidirectional only (can't traverse backward

→ code for singly linked list

```
#include <stdio.h>
#include <stdlib.h>
struct Node {
 int data;
 struct Node * next;
};
struct Node * head = NULL;

void insertAtHead (int val) {
struct Node * newNode = (struct Node*) malloc (
                        sizeof (struct Node));
newNode → data = val;
newNode → next = head;
head = newNode;
printf ("Inserted %d at beggining", val);
}

void insertAtTail (int val) {
struct Node * newNode = (struct Node*) malloc (sizeof (
                                struct Node));
newNode → data = val;
newNode → next = NULL;
if (head == NULL) {
 head = newNode;
 return;
 }
struct Node * temp = head;
while (temp → next != NULL) {
 temp = temp → next;
 }
```

pty List

```
temp → next = newNode;
printf ("%d inserted at end", val);
}

void insertAtPosition (int val, int pos) {
if (pos == 1) {
insertAtHead (val);
return;
}
struct Node * newNode = (struct Node*) malloc (sizeof(
struct Node));
newNode → data = val;
struct Node * temp = head;
for (int i = 1; temp != NULL && i < pos-1; i++) {
temp = temp → next;
}
if (temp == NULL) {
printf ("invalid position");
free (newNode);
return;
}
newNode → next = temp → next;
temp → next = newNode;
printf ("%d inserted at position %d", val, pos);
}

void deleteFromHead () {
if (head == NULL) {
printf ("List is empty");
return;
}
```

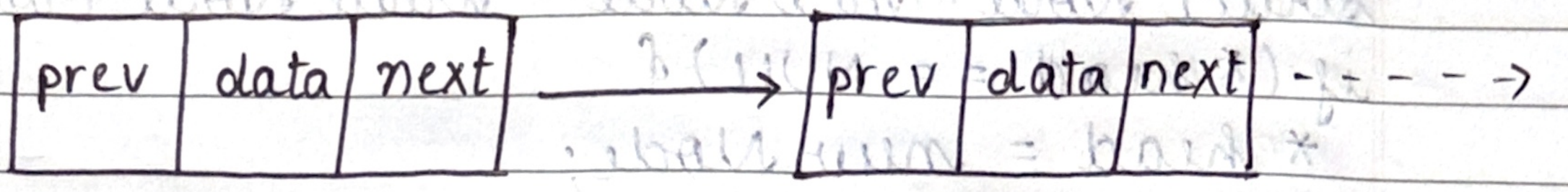
```
struct Node * temp = head;
head = head → next;
printf ("%d deleted from begginning", temp→data);
free (temp);
}
void deleteFromTail() {
if (head == NULL) {
printf ("List is empty");
return;
}
if (head → next == NULL) {
printf ("%d deleted from end", head → data);
free (head);
head = NULL;
return;
}
struct Node * temp = head;
while (temp → next → next != NULL) {
temp = temp → next;
}
printf ("%d deleted from end", temp → next → data);
free (temp → next);
temp → next = NULL;
}

void deleteFromPos (int pos) {
if (head == NULL) {
printf ("List is empty");
return;
}
struct Node * temp = head;
```

```
for (int i = 1; temp != NULL && i < pos - 1; i++) {
    temp = temp -> next;
}
if (temp == NULL || temp -> next == NULL) {
printf ("invalid position");
return;
}
struct Node * nodeToDelete = temp -> next;
temp -> next = nodeToDelete -> next;
printf ("%d deleted from %d position", nodeToDelete -> data,
                                        pos);
free (nodeToDelete);
}

void display () {
if (head == NULL) {
printf ("List is empty");
return;
}
struct Node * temp = head;
printf ("List : ");
while (temp != NULL) {
printf ("%d -> ", temp -> data);
temp = temp -> next;
}
printf ("NULL");
}
int main () {
insertAtHead (10);
insertAtEnd (9);
deleteFromHead();
display ();
}
```

→ Doubly linked list




```
# include <stdio.h>
Struct Node {
int data;
Struct Node* prev;
Struct Node* next;
};
```

→ Function to create new Node

```
Struct Node* create Node (int val) {
Struct Node* new Node = (struct Node*) malloc (size
                                of (struct Node);
new Node → data = val;
new Node → prev = NULL;
new Node → next = NULL;
return new Node;
}

void insert At Head (struct Node** head, int val)
{
struct Node* new Node = create Node (val);
if (* head != NULL) {
new Node → next = * head;
(* head) → prev = new Node;
}
* head = new Node;
}
```

```
void insertAtEnd(struct Node**head,int val)
{
struct Node* newNode = createNode(val);
if(*head==NULL){
 *head = newNode;
 }
else{
struct Node * temp = *head;
while(temp→next!=NULL){
temp = temp→next;
 }
temp→next = newNode;
newNode→prev = temp;
}}
void insertAtPosition(struct Node**head,int val,
                      int pos){
if(pos==1){
insertAtHead(head,val);
return;
}
struct Node* temp = *head;
for(int i=1;temp!=NULL && i<pos-1;i++){
 temp = temp→next;
}
if(temp==NULL){
return;
}
Struct Node* New Node = createNode(val);
new Node→next=temp→next;
new Node→prev = temp;
if(temp→next!=NULL){
```

```
temp → next → prev = newNode;
}
temp → next = newNode;
}
```

```
void deleteFromHead (struct Node **head){
if (*head == NULL){
return;
}
struct Node * temp = *head;
*head = (*head) → next;
if (*head != NULL) {
  (*head) → prev = NULL;
}
free (temp);
}
```

```
void deleteFromEnd (struct Node **head) {
if (*head == NULL) {
return;
}
struct Node * temp = *head;
if (temp → next == NULL) {
  *head = NULL;
free (temp);
return;
}
while (temp → next != NULL") {
 temp = temp → next;
}
temp → prev → next = NULL;
 free (temp);
}
```

```
void deleteAtPosition (struct Node **head,
int pos) {
if (head == NULL) {
return;
}
if (pos == 1) {
deleteFromHead (head);
return;
}
struct Node * temp = * head;
for (int i = 1; temp != NULL && i < pos; i++) {
 temp = temp → next;
}
if (temp == NULL) { return ; }
if (temp → next != NULL) {
 temp → next → prev = temp → prev;
}
if (temp → prev != NULL) {
temp → prev → next = temp → next;
}
free temp;
}

void display (struct Node * head) {
struct Node * temp = head;
while (temp != NULL) {
printf ("%d ↔ ", temp → data);
}
printf ("NULL");
}
```

```
int main() {
Struct Node * head = NULL;
insert At Head (& head , temp);
    .
    .
    .
    .
return 0;
 }
```

→ Applications of Linked Lists

1. sprating Systems
   Manages RAM block allocation, process schedulin queues & FAT file system disc tracks
2. Navigation & State history
   Powers undo/redo actions in editors, Back/ forward control in browsers & photo gallery navigation
3. Media Players
   Manages audio/video playlist with next or previous traversal & seamless circular looping
4. Building other Data Structures
   Acts as foundation for stacks, queues, graph adjacency lists and hash table collision handling.
5. High performance caching
   Pairs with hash maps to build least recently used caches with O(1) updates & eviction
6. Mathematical spirations
   Efficiently stores sparse matrices & handles polynomial addition/multiplication
7. Core Advantage
   Provides dynamic memory sizing & fast

O(1) insertions / deletions without array element shifting

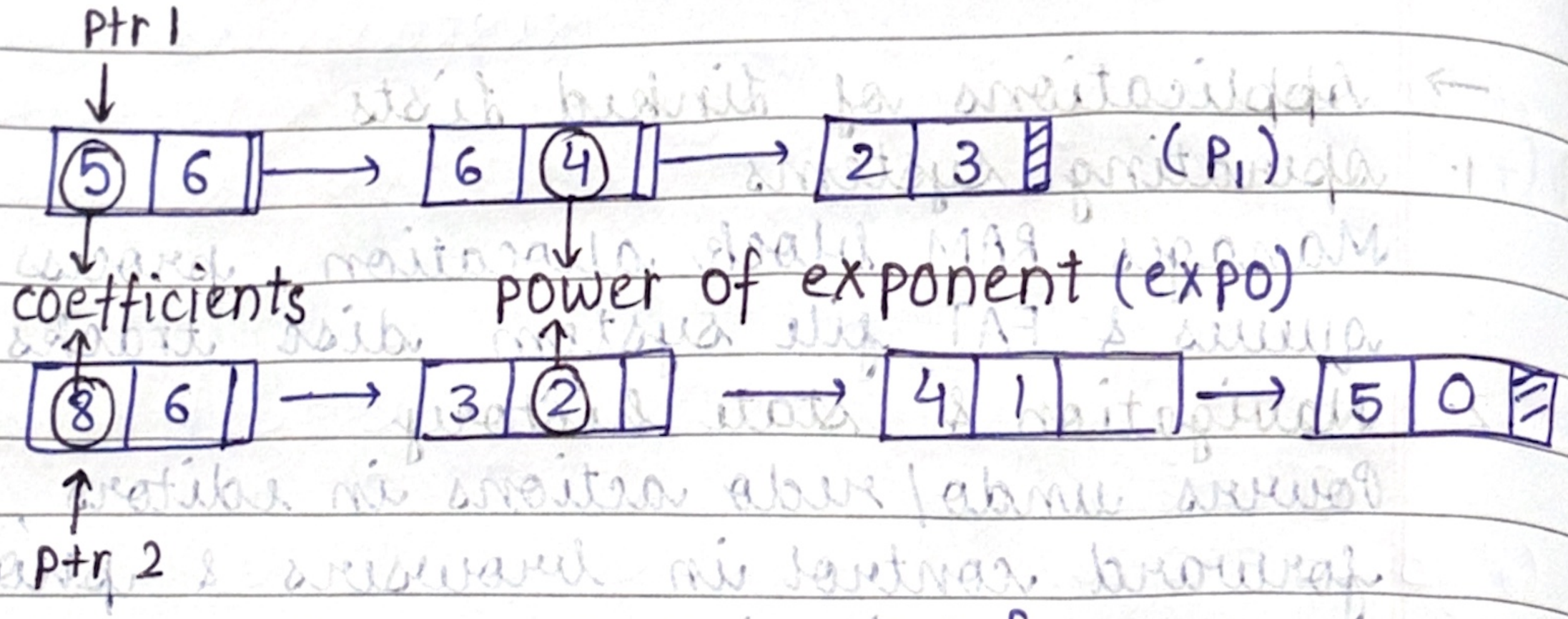
→ Polynomial Addition

- Application of linked lists

eg :-

$5x^6 + 6x^4 + 2x^3 \rightarrow P_1$

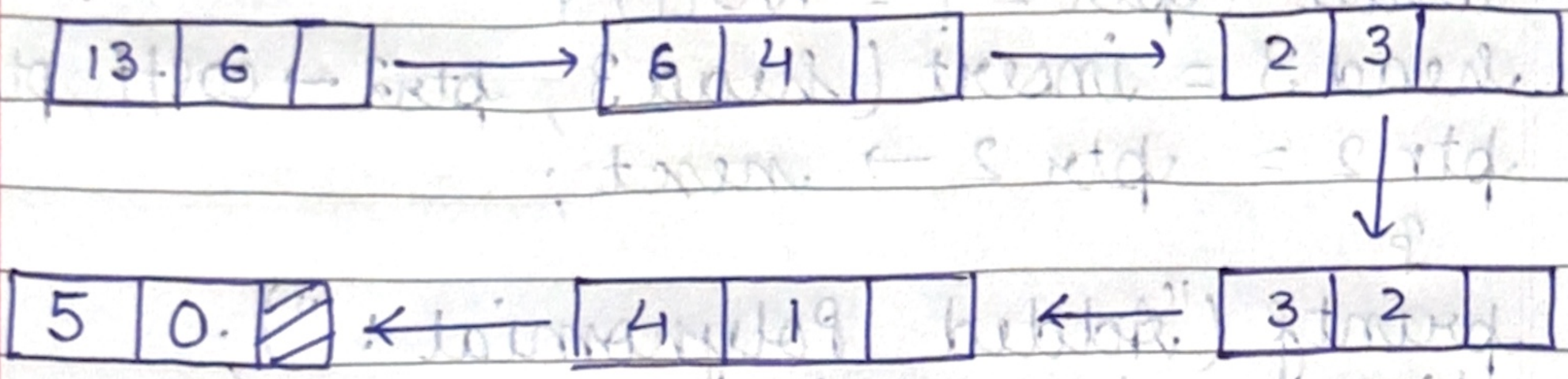
$8x^6 + 3x^2 + 4x + 5 \rightarrow P_2$



1. If (ptr 1 → expo == ptr 2 → expo) {
   Add the coefficients & insert newly created node in the resultant linked list & make ptr 1 & ptr 2 point to next nodes

2. If (ptr 1 → expo > ptr 2 → expo) {
   Insert the node pointed by ptr 1 in the resultant linked list and make ptr 1 point to next nodes

3. If (ptr 1 → expo < ptr 2 → expo) {
   Insert the node pointed by ptr 2 in the resultant linked list and make ptr 2 point to next node




```
void polyAdd (struct node* head1, struct node
                                    * head2){
struct Node* ptr1 = head1;
struct Node* ptr2 = head2;
struct Node* head3 = NULL;
while (ptr1 != NULL && ptr2 != NULL){
if (ptr1 → expo == ptr2 → expo){
head3 = insert (head3, ptr1 → coeff + ptr2 → coeff,
                        ptr1 → expo);
ptr1 = ptr1 → next;
ptr2 = ptr2 → next;
  }
else if (ptr1 → expo > ptr2 → expo){
head3 = insert (head3, ptr1 → coeff, ptr1 → expo);
ptr1 = ptr1 → next;
  }
else if (ptr1 → expo < ptr2 → expo){
head3 = insert (head3, ptr2 → coeff, ptr2 → expo);
ptr2 = ptr2 → next;
  }
}
while (ptr1 != NULL){
head3 = insert (head3, ptr1 → coeff, ptr1 → expo);
ptr1 = ptr1 → next;
  }
```

```
while (ptr2 != NULL) {
head3 = insert (head3, ptr2 → coeff, ptr2 → expo);
ptr2 = ptr2 → next;
}
printf ("Added Polynomial : ");
display (head3);
}
```

→ Sparse matrix

A matrix (2D array) : having maximum entries as 0

eg :-

$$\begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 4 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}_{4\times4}$$

→ not efficient : memory waste as we store 0 (16 X 2 = 32 byte)

→ Array Representation

	0	1	2
row	1	2	3
col	2	0	1
val	1	4	1

(9 X 2 = 18 bytes)

takes comparitively less memory when large matrix

→ Linked-List Representation

